DSA PROBLEMS:
Analysis of Common Techniques

Box Pattern

Difficulty: **Easy** Accuracy: **55.52%** Submissions: **634+** Points: **2**

Given **N**, print the pattern as given in the example.

Input Format:

The first line of the input contains a single integer T denoting the number of test cases.

First line of each test case contains one integer N.

Output Format:

For each test case, in a new line, print the given pattern as shown in the example.

Example 1:

Input:

1

4

Output:

1 1 1 1 1 1 1

1 2 2 2 2 2 1

1 2 3 3 3 2 1

1 2 3 4 3 2 1

1 2 3 3 3 2 1

1 2 2 2 2 2 1

1 1 1 1 1 1 1

Constraints:

1 <= T <= 100

1 <= N <= 100

Array of alternate +ve and -ve no.s

Difficulty: **Basic**Accuracy: **17.42%**Submissions: **2K+**Points: **1**

Given an unsorted array of positive and negative numbers. Your task is to create an array of alternate positive and negative numbers without changing the relative order of positive and negative numbers.

Note : If any one of the postive or negative numbers gets finished early while forming the result array than place rest of the remaining elements at the end in the relative order.

Note: Consider 0 as a positive number, and Array should start with positive number, for this problem.

Input:

First line of input contains an integer **T** denoting the number of test cases. For each testcase, first line contains N, size of array. The subsequent line contains N array elements.

Output:

Print the modified array.

Example:

Input:

1

9

9 4 -2 -1 5 0 -5 -3 2

Output:

9 -2 4 -1 5 -5 0 -3 2

Constraints:

$1 \le T \le 100$

$1 \le N \le 10^7$

$-10^6 \le a[] \le 10^7$

The sum of N for overall test cases does not exceeds 10^7.

Pairs of Adjacent elements

Difficulty: **Easy**Accuracy: **82.02%**Submissions: **804+**Points: **2**

Given an array **arr[]** of positive integers of size **N**. The task is to count pairs of adjacent elements which are consecutive. A pair is said to be consecutive if the second element in the pair is greater than the first element by 1. For example, (4,5) is a pair of consecutive elements.

Example 1:

Input: N = 5

arr = {5, 7, 6, 7, 4}

Output: 1

Explaination: (6, 7) pair is the only such pair.

Example 2:

Input: N = 4

arr = {1, 2, 3, 4}

Output: 3

Explaination: (1, 2), (2, 3), (3, 4) are the pairs.

Your Task:

You do not need to read input or print anything. Your task is to complete the function **adjacentPairs()** which takes N and arr as input parameters and returns the number of pairs which satisfies the condition.

Expected Time Complexity: O(N)

Expected Auxilary Space: O(1)

Constraints:

$1 \le N \le 1000$

$1 \le arr[i] \le 10^5$

Arrays

Array insert at end

Difficulty: **Basic** Accuracy: **87.65%** Submissions: **55K+** Points: **1**

Insertion is a basic but frequently used operation. Arrays in most languages can not be dynamically shrinked or expanded. Here, we will work with such arrays and try to insert an element at the end of the array.

You need to modify the given array **arr**. The size of the array is given by **sizeOfArray**. You need to **insert** an **element** at the **end.** Array already have the sizeofarray -1 elements. Find Kth Rotation

Examples :

Input: sizeOfArray = 6, arr[] = {1, 2, 3, 4, 5}, element = 90
Output: 1 2 3 4 5 90
Explanation: After inserting 90 at the end, we have array elements as 1 2 3 4 5 90.
Input: sizeOfArray = 4, arr[] = {1, 2, 3}, element = 50
Output: 1 2 3 50
Explanation: After inserting 50 at the end, we have array elements as 1 2 3 50.

Your Task:

You don't need to read input or print anything. You only need to complete the function **insertAtEnd()** that takes arr, sizeOfArray, element as input and modifies the given array **arr** as per requirements. The **driver code** takes care of the **printing**.

Expected Time Complexity: $O(1)$.
Expected Auxiliary Space: $O(1)$.

Constraints:

1 <= sizeOfArray <= 1000

0 <= element, arr_i <= 10^6

Stock buy and sell

Difficulty: **Medium**Accuracy: **29.18%**Submissions: **272K+**Points: **4**

The cost of stock on each day is given in an array **A**[] of size **N**. Find all the segments of days on which you buy and sell the stock such that the sum of difference between sell and buy prices is maximized. Each segment consists of indexes of two elements, first is index of day on which you buy stock and second is index of day on which you sell stock.

Note: Since there can be multiple solutions, the driver code will print 1 if your answer is correct, otherwise, it will return 0. In case there's no profit the driver code will print the string "**No Profit**" for a correct solution.

Example 1:

Input:

N = 7

A[] = {100,180,260,310,40,535,695}

Output:

1

Explanation:

One possible solution is (0 3) (4 6)

We can buy stock on day 0,

and sell it on 3rd day, which will

give us maximum profit. Now, we buy

stock on day 4 and sell it on day 6.

Example 2:

Input:

N = 5

A[] = {4,2,2,2,4}

Output:

1

Explanation:

There are multiple possible solutions.
one of them is (3 4)
We can buy stock on day 3,
and sell it on 4th day, which will
give us maximum profit.

Your Task:

The task is to complete the function **stockBuySell()** which takes an array of A[] and N as input parameters and finds the days of buying and selling stock. The function must return a 2D list of integers containing all the buy-sell pairs i.e. the first value of the pair will represent the day on which you buy the stock and the second value represent the day on which you sell that stock. If there is No Profit, return an empty list.

Expected Time Complexity: O(N)

Expected Auxiliary Space: O(N)

Constraints:

$2 \leq N \leq 10^6$

$0 \leq A[i] \leq 10^6$

Array insert at index

Difficulty: **Basic**Accuracy: **44.81%**Submissions: **97K+**Points: **1**

Insertion is a basic but frequently used operation. Arrays in most languages cannnot be dynamically shrinked or expanded. Here, we will work with such arrays and try to insert an element at some index.

You are given an array **arr**(0-based index). The size of the array is given by **sizeOfArray**. You need to **insert** an **element** at given **index**.

Example 1:

Input:
sizeOfArray = 6
arr[] = {1, 2, 3, 4, 5}
index = 5, element = 90
Output: 1 2 3 4 5 90
Explanation: 90 is inserted at index 5(0-based indexing). After inserting, array elements are like 1, 2, 3, 4, 5, 90.

Example 2:

Input:
sizeOfArray = 6
arr[] = {1, 2, 3, 4, 5}
index = 2, element = 90
Output: 1 2 90 3 4 5
Explanation: 90 is inserted at index 2(0-based indexing). After inserting, array elements are like 1, 2, 90, 3, 4, 5.

Your Task:

You don't need to read input or print anything.. The input is already taken care of by the driver code. You only need to complete the function **insertAtIndex()** that takes *arr*, *sizeOfArray*, *index*, *element* as input and modifies the array *arr* as per requirements. The **printing** is done by **driver code**.

Expected Time Complexity: O(N).
Expected Auxiliary Space: O(1).

Constraints:

1 <= sizeOfArray <= 10000

0 <= element, arr_i <= 10^6

0 <= index <= sizeOfArray-1

Rearrange an array with O(1) extra space

Difficulty: **Medium**Accuracy: **56.34%**Submissions: **115K+**Points: **4**

Given an array **arr[]** of size **N** where every element is in the range from **0 to n-1**. Rearrange the given array so that the transformed array **arr^T[i]** becomes **arr[arr[i]]**.

NOTE: arr and **arr^T** are both same variables, representing the array before and after transformation respectively.

Example 1:

Input:

N = 2

arr[] = {1,0}

Output: 0 1

Explanation:

arr[arr[0]] = arr[1] = 0

arr[arr[1]] = arr[0] = 1

So, arr^T becomes {0, 1}

Example 2:

Input:

N = 5

arr[] = {4,0,2,1,3}

Output: 3 4 2 0 1

Explanation:

arr[arr[0]] = arr[4] = 3
arr[arr[1]] = arr[0] = 4
arr[arr[2]] = arr[2] = 2
arr[arr[3]] = arr[1] = 0
arr[arr[4]] = arr[3] = 1
and so on
So, arr^T becomes {3, 4, 2, 0, 1}

Your Task:
You don't need to read input or print anything. The task is to complete the function **arrange**() which takes arr and N as input parameters and rearranges the elements in the array in-place.

Expected Time Complexity: O(N)
Expected Auxiliary Space: O(1)

Constraints:
1 <= N <= 10^5
0 <= Arr[i] < N

Array Leaders
Difficulty: **Easy**Accuracy: **29.94%**Submissions: **707K+**Points: **2**
Given an array **arr** of **n** positive integers, your task is to find all the leaders in the array. An element of the array is considered a leader if it is greater than all the elements on its right side or if it is equal to the maximum element on its right side. The rightmost element is always a leader.

Examples

Input: n = 6, arr[] = {16,17,4,3,5,2}
Output: 17 5 2
Explanation: Note that there is nothing greater on the right side of 17, 5 and, 2.
Input: n = 5, arr[] = {10,4,2,4,1}

Output: 10 4 4 1

Explanation: Note that both of the 4s are in output, as to be a leader an equal element is also allowed on the right. side

Input: n = 4, arr[] = {5, 10, 20, 40}

Output: 40

Explanation: When an array is sorted in increasing order, only the rightmost element is leader.

Input: n = 4, arr[] = {30, 10, 10, 5}

Output: 30 10 10 5

Explanation: When an array is sorted in non-increasing order, all elements are leaders.

Expected Time Complexity: O(n)

Expected Auxiliary Space: O(n)

Constraints:

$1 <= n <= 10^7$

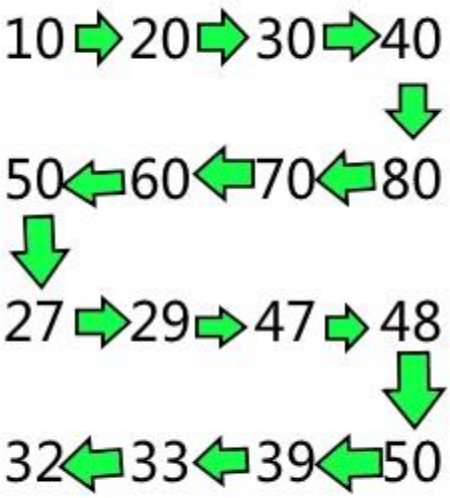
$0 <= arr[i] <= 10^7$

Matrix

Print Matrix in snake Pattern

Difficulty: **Easy**Accuracy: **68.08%**Submissions: **71K+**Points: **2**

Given a **matrix** of size **N x N**. Print the elements of the matrix in the snake like pattern depicted below.



Output: 10 20 30 40 80 70 60 50 27 29 47 48 50 39 33 32

Example 1:

Input:
N = 3
matrix[][] = {{45, 48, 54},
{21, 89, 87}
{70, 78, 15}}
Output:
45 48 54 87 89 21 70 78 15
Explanation:
Matrix is as below:
45 48 54
21 89 87
70 78 15
Printing it in snake pattern will lead to

the output as 45 48 54 87 89 21 70 78 15.

Example 2:

Input:
N = 2
matrix[][] = {{1, 2},
{3, 4}}
Output:
1 2 4 3
Explanation:
Matrix is as below:
1 2
3 4
Printing it in snake pattern will
give output as 1 2 4 3.

Your Task:
You dont need to read input or print anything. Complete the function **snakePattern()** that takes matrix as input parameter and returns a list of integers in order of the values visited in the snake pattern.

Expected Time Complexity: O(N * N)
Expected Auxiliary Space: O(N * N) for the resultant list only.

Constraints:
$1 <= N <= 10^3$
$1 <= mat[i][j] <= 10^9$

Boundary traversal of matrix

Difficulty: **Easy**Accuracy: **35.32%**Submissions: **102K+**Points: **2**

You are given a matrix of dimensions **n x m**. The task is to perform boundary traversal on the matrix in a clockwise manner starting from the first row of the matrix.

Example 1:

Input:
n = 4, m = 4
matrix[][] = {{1, 2, 3, 4},
{5, 6, 7, 8},
{9, 10, 11, 12},
{13, 14, 15,16}}
Output: 1 2 3 4 8 12 16 15 14 13 9 5
Explanation:
The matrix is:
1 2 3 4
5 6 7 **8**
9 10 11 **12**
13 14 15 16
The boundary traversal is:
1 2 3 4 8 12 16 15 14 13 9 5

Example 2:

Input:
n = 3, m = 4
matrrix[][] = {{12, 11, 10, 9},
{8, 7, 6, 5},
{4, 3, 2, 1}}
Output: 12 11 10 9 5 1 2 3 4 8

Your Task:
Complete the function **boundaryTraversal()** that takes matrix, n and m as input parameters and returns the list of integers that form the boundary traversal of the matrix in a clockwise manner.

Expected Time Complexity: $O(N + M)$

Expected Auxiliary Space: $O(N+M)$

Constraints:

$1 <= n, m <= 1000$

$0 <= matrix_i <= 1000$

Transpose of Matrix

Difficulty: **Easy**Accuracy: **66.5%**Submissions: **100K+**Points: **2**

Write a program to find the **transpose** of a square matrix of size N*N. Transpose of a matrix is obtained by changing rows to columns and columns to rows.

Example 1:

Input:

N = 4

mat[][] = {{1, 1, 1, 1},
{2, 2, 2, 2}
{3, 3, 3, 3}
{4, 4, 4, 4}}

Output:

{{1, 2, 3, 4},
{1, 2, 3, 4}
{1, 2, 3, 4}
{1, 2, 3, 4}}

Example 2:

Input:

N = 2

mat[][] = {{1, 2},
{-9, -2}}

Output:

{{1, -9},

{2, -2}}

Your Task:
You dont need to read input or print anything. Complete the function **transpose**() which takes matrix[][] and N as input parameter and finds the transpose of the input matrix. You need to do this in-place. That is you need to update the original matrix with the transpose.

Expected Time Complexity: O(N * N)
Expected Auxiliary Space: O(1)

Constraints:
$1 <= N <= 10^3$
$-10^9 <= mat[i][j] <= 10^9$

Recursion

Print Array Elements Using Recursion

Difficulty: **Basic**Accuracy: **56.49%**Submissions: **13K+**Points: **1**

You are given an array arr of size n. You need to print the array elements from start to end using given recursive function.

Example 1:

Input:
n = 5
arr[] = {1,2,3,4,5}
Output: 1 2 3 4 5

Example 2:

Input:

n = 4
arr[] = {5,4,2,1}
Output: 5 4 2 1

Your Task:
Complete the function **printArrayRecursively()** that takes array and size n as parameters and prints the array elements recursively. The newline is provided by driver code.

Expected Time Complexity: O(N).
Expected Auxiliary Space: O(N) (Recursive).

Constraints:
1 <= n <= 100

Power Using Recursion
Difficulty: **Basic**Accuracy: **75.78%**Submissions: **41K+**Points: **1**
You are given **two numbers n and p**. You need to find n^p.

Example 1:

Input:
n = 9 p = 9
Output: 387420489
Explanation: 387420489 is the value
obtained when 9 is raised to the
power of 9.

Example 2:

Input:

n = 2 p = 9

Output: 512

Explanation: 512 is the value obtained when 2 is raised to the power of 9.

Your Task:

You don't need to read input or print anything. You only need to complete the function **RecursivePower()** that takes **n** and **p** as **parameters** and **returns** n^p.

Expected Time Complexity: O(p).

Expected Auxiliary Space: O(p).

Constraints:

1 <= n <= 9

0 <= p <= 9

Hashing

Linear Probing in Hashing

Difficulty: **Easy**Accuracy: **25.49%**Submissions: **48K+**Points: **2**

Linear probing is a collision-handling technique in hashing. Linear probing says that whenever a collision occurs, search for the immediate next position.

Given an array of integers and a hash table size. Fill the array elements into a hash table using **Linear Probing** to handle collisions. Duplicate elements must be mapped to the same position in the hash table while colliding elements must be mapped to the [(value+1)%hashSize] position.

Note :- If there's no more space to insert a new element, just **drop** that element.

Example 1:

Input:

hashSize = 10

sizeOfArray = 4
Array[] = {4,14,24,44}

Output:

-1 -1 -1 -1 4 14 24 44 -1 -1

Explanation: 4%10=4. So put 4 in hashtable[4].Now, 14%10=4, but hashtable[4] is alreadyfilled so put 14 in the next slot and so on.

Example 2:

Input:

hashSize = 10
sizeOfArray = 4
Array[] = {9,99,999,9999}

Output:

99 999 9999 -1 -1 -1 -1 -1 -1 9

Explanation: 9%10=9. So put 9 in hashtable[9]. Now, 99%10=9, but hashtable[9] is already filled so put 99 in the (99+1)%10 =0 slot so 99 goes into hashtable[0] and so on.

Your Task:

You don't need to read input or print anything. Your task is to complete the function **linearProbing()** which takes the hash table size (HashSize), an integers array arr[], and its size N as input parameters and inserts all the elements of the array arr[] into a hash table. The function should return the hash table.
The empty cells of the hash table are to be given a value of -1.
Also, if there's no more space to insert a new element, just **drop** that element.

Expected Time Complexity: O(N)

Expected Auxiliary Space: O(N)

Constraints:

1 <= hashSize <= 1000

1 <= sizeOfArray <= 10000

0 <= Array[] <= 10^5

Separate chaining in Hashing

Difficulty: **Easy**Accuracy: **64.96%**Submissions: **16K+**Points: **2**

Separate chaining technique in hashing allows to us to use a linked list at each hash slot to handle the problem of collisions. That is, every slot of the hash table is a linked list, so whenever a collision occurs, the element can be appened as a node to the linked list at the slot.

In this question, we'll learn how to fill up the hash table using ***Separate chaining*** **technique**. Given an array (consisting of distinct integers) and a hashtable size, you have to fill the elements of the array into a hash table of given size.

Examples :

Input: hashSize = 10 sizeOfArray = 6 arr[] = {92,4,14,24,44,91}

Output:

1->91

2->92

4->4->14->24->44

Explanation: 92%10=2 so 92 goes to slot 2. 4%10=4 so 4 goes to slot 4. 14%10=4. But 4 is already occupied so we make a linked list at this position and add 14 after 4 in slot 4 and so on.

Input: hashSize = 10 sizeOfArray = 5 arr[] = {12,45,36,87,11}

Output:

1->11

2->12
5->45
6->36
7->87

Explanation: 12%10=2 so 12 goes to slot 2. 45%10=5 goes to slot 5. 36%10=6 goes to slot 6. 87%10=7 goes to slot 7 and finally. 11%10=1 goes to slot 1.

Your Task:

You need to complete the function **separateChaining** that takes **hashSize, arr, and sizeOfArr** as parameters, inserts elements of arr in the hashTable at positions by using **arr[i]%hashSize** and then returns the hashTable. The printing is done automatically by the driver code.

Expected Time Complexity: O(N).

Expected Auxiliary Space: O(N).

Constraints:

$2 <= hashSize <= 10^3$

$1 <= sizeOfArray <= 10^3$

$0 <= arr_i <= 10^7$

Quadratic Probing in Hashing

Difficulty: **Easy**Accuracy: **53.42%**Submissions: **23K+**Points: **2**

Quadratic probing is a collision handling technique in hashing. Quadratic probing says that whenever a collision occurs, search for i^2 position.

Given an array of integers and a Hash table. Fill the elements of the array into the hash table by using Quadratic Probing in case of collisions.

Example 1:

Input:

hashSize = 11

N = 4

Array[] = {21,10,32,43}

Output:

10 -1 -1 32 -1 -1 -1 -1 43 -1 21

Explanation: 21%11=10 so 21 goes into hashTable[10] position. 10%11=10. hashTable[10] is already filled so we try for $(10+1^2)\%11=0$ position. hashTable[0] is empty so we put 10 there. 32%11=10. hashTable[10] is filled. We try $(32+1^2)\%11=0$. But hashTable[0] is also already filled. We try $(32+2^2)\%11=3$. hashTable[3] is empty so we put 32 in hashTable[3] position. 43 uses $(43+3^2)\%11=8$. We put it in hashTable[8].

Example 2:

Input:

hashSize = 11

N = 4

Array[] = {880,995,647,172 }

Output:

880 -1 -1 -1 -1 995 -1 172 -1 647 -1

Explanation: Using the similar approach as used in above explanation we will get the output like

880 -1 -1 -1 -1 995 -1 172 -1 647 -1.

Example 3:

Input:

hashSize = 11

N = 4

Array[] = {4,4,4,4}

Output: -1 -1 -1 -1 4 -1 -1 -1 -1 -1 -1

Your Task:

You don't need to read input or print anything. Your task is to complete the function **QuadraticProbing()** which takes the hash table **hash[]**, the hash table size **hashSize**, an array **arr[]** and the size of the array **N** as inputs and inserts all the elements of the array arr[] into the hash table using Quadratic Probing as a collision handling technique.

Note: You need to map duplicate elements incase, they have the same hash value even after quadratic probing.

Expected Time Complexity: $O(N)$.

Expected Auxiliary Space: $O(1)$.

Constraints:

2 <= hashSize **(prime)** <= 97

1 <= N < hashSize*0.5

0 <= Array[] <= 10^5

Note:

All the positions that are unoccupied are denoted by -1 in the hash table.

An empty slot can only be found if **load factor < 0.5** and hash table size is a **prime number**.

The **given testcases satisfy the above condition** so you can **assume** that an empty slot is always reachable.

Count Non-Repeated Elements

Difficulty: **Easy**Accuracy: **65.14%**Submissions: **26K+**Points: **2**

Hashing is very useful to keep track of the frequency of the elements in a list.

You are given an array of integers. You need to print the count of non-repeated elements in the array.

Example 1:

Input:
10
1 1 2 2 3 3 4 5 6 7

Output:
4

Explanation:
4, 5, 6 and 7 are the
elements with frequency 1 and rest
elements are repeated so the number
of non-repeated elements are 4.

Example 2:

Input:
5
10 20 30 40 10

Output:
3

Explanation:
20, 30, 40 are the
elements with the frequency 1 and
10 is the repeated element to
number of non-repeated elements
are 3.

Your Task:
You don't need to read input or print anything. You only need to complete the

function **countNonRepeated()** that takes array **arr[]** and its
size **n** as **parameters** and **returns** the count of non-repeating elements in the array.

Expected Time Complexity: O(n).
Expected Auxiliary Space: O(n).

Constraints:
$1 <= n <= 10^3$
$0 <= arr_i <= 10^7$

Print Non-Repeated Elements

Difficulty: **Easy**Accuracy: **46.54%**Submissions: **39K+**Points: **2**

Hashing is very useful to keep track of the frequency of the elements in a list.

You are given an array of integers. You need to print the non-repeated elements as they appear in the array.

Example 1:

Input:
n = 10
arr[] = {1,1,2,2,3,3,4,5,6,7}
Output: 4 5 6 7
Explanation: 4, 5, 6 and 7 are the only elements which is having only 1 frequency and hence, Non-repeating.

Example 2:

Input:
n = 5
arr[] = {10,20,40,30,10}
Output: 20 40 30
Explanation: 20, 40, 30 are the only elements which is having only 1

frequency and hence, Non-repeating.

Your Task:

You don't need to read input or print anything. You only need to complete the function **printNonRepeated()** that takes **arr and n as parameters** and return the array which has the distinct elements in same order as they appear in input array. The newline is **appended** automatically by the **driver code**.

Expected Time Complexity: $O(n)$.

Expected Auxiliary Space: $O(n)$.

Constraints:

$1 <= n <= 10^3$

$0 <= arr_i <= 10^7$

First Repeating Element

Difficulty: **Easy**Accuracy: **38.8%**Submissions: **415+**Points: **2**

Given an integer array. The task is to find the first repeating element in the array i.e., an element that occurs more than once and whose index of first occurrence is smallest.

Input :

The first line contains an integer T denoting the total number of test cases. In each test cases, First line is number of elements in array N and second line contains N space separated integer values of the array.

Output:

In each separate line print the index of first repeating element, if there is not any repeating element then print -1 (without quotes). Use **1** Based Indexing.

Constraints:

$1 <= T <= 500$

$1 <= N <= 10^6$

$0 <= A_i <= 10^6$

Example:

Input:

3
7
1 5 3 4 3 5 6
4
1 2 3 4
5
1 2 2 1 3

Output:

2
-1
1

Explanation:

Testcase 1: 5 is appearing twice and its first appearence is at index 2 which is less than 3 whose first occuring index is 3.

Testcase 2: None of the elements are occuring twice . So, the answer is -1.

Testcase 3: 1 is appearing twice and its first appearence is at index 1 which is less than 2 whose first occuring is at index 2.

Candy Run

Difficulty: **Easy**Accuracy: **34.12%**Submissions: **387+**Points: **2**

Phineas and Ferb are very smart and can think of very efficient algorithms while solving problems. There are **N** candies placed on the road from left to right at one meter apart from each other. Each candy is represented by a distinct number **perm[i]** in range **[1, N]**. To reach a particular candy and eventually eat it, Phineas thinks it is more efficient to start running from left side while Ferb thinks it is more efficient to start running from right side. But their elder sister Candice knows that she is the smartest and claims that both approaches are equivalent. So she decided to give them **Q** queries, each query **b[i]** consist of a

distinct candy number. For each query Phineas and Ferb have to reach that candy by following their own approaches. After all queries are performed sum of distances ran by Phineas and Ferb is calculated and the person whose approach was correct is declared smartest. For each test case, you have to calculate total length covered by Phineas and Ferb and find out who is the smartest person.

Input:

1. The first line of the input contains a single integer T denoting the number of test cases. The description of T test cases follows.
2. The first line of each test case contains two-space separated integers N and Q.
3. The second line of each test case contains N space-separated positive integers which represents permutation of N.
4. The third line of each test case contains Q space-separacted positive integers which represents queries.

Output: For each test case, print two integers - length covered by Phineas and Ferb. After printing two integers print the name of the person who you think is the smartest in the same line.

Constraints:

1. $1 <= T <= 10^5$
2. $1 <= N, Q <= 10^6$
3. $1 <= perm[i], b[i] <= N$
4. It is guaranteed that sum of N and Q over all testcases does not exceed 10^6.

Example:

Input:
2
4 3
3 1 2 4
2 4 2
1 1
1
1
Output:
10 5 Ferb
1 1 Candice
Explanation:
In the first test case, total length covered by Phineas over all queries is 3 + 4 + 3 = 10 and total length covered by Ferb over all queries is 2 + 1 + 2 = 5 respectively. Since Ferb's algorithm i.e. to start running from right end is more efficient, he is declared the smartest.

String

Check Palindrome

Difficulty: **Easy**Accuracy: **49.98%**Submissions: **19K+**Points: **2**

You are given a number n. You need to see if the number is a palindrome or not (recursively)

Example 1:

Input:
n = 100
Output: 0

Example 2:

Input:
n = 101
Output: 1

Your Task:

Complete the function isPalin that takes n as parameter and returns true or false . (In case of true, 1 will be printed by the driver code, otherwise 0)

Expected Time Complexity: O(Log(N)).
Expected Auxiliary Space: O(Log(N)) (Recursive).

Constraints:
$1 <= n <= 10^8$

Check if a String is Subsequence of Other

Difficulty: **Medium**Accuracy: **51.68%**Submissions: **16K+**Points: **4**

Given two strings **A** and **B**, find if **A** is a subsequence of **B**. A subsequence is a sequence that can be derived from another sequence by deleting some elements without changing the order of the remaining elements.

Example 1:

Input:
A = AXY
B = YADXCP
Output: False
Explanation: A is not a subsequence of B
as 'Y' appears before 'A'.

Example 2:

Input:
A = gksrek
B = geeksforgeeks
Output: True
Explanation: A is a subsequence of B.

Your Task:
You dont need to read input or print anything. Complete the function **isSubSequence()** which takes A and B as input parameters and returns a boolean value denoting if A is a subsequence of B or not.

Expected Time Complexity: O(N) where N is length of string B.
Expected Auxiliary Space: O(1)

Constraints:
$1 <= |A|,|B| <= 10^6$

Check Substring

Difficulty: **Basic** Accuracy: **45.46%** Submissions: **506+** Points: **1**

Given two strings A and B, check whether B is a substring of A or Not.

Input: Test case should contain (2*N+1) lines.First line contains number of test cases N. and then next 2 lines for each test contains strings A and B.

Output: Print the leftmost starting index of B, if B is a substring of A else print -1 for each test case.

Example 1:

Input:
2
GeeksForGeeks
Geeks
Hello World
World
Output:
0
7
Explanation: left most starting index of Geeks is 0.
left most starting index of World is 6.

Example 2:

Input:
1
Welcome to python
pandas
Output:
-1
Explanation: Because pandas is not present.

Your Task:
Your task is to read two line of input A and B for each test case, and then print the leftmost starting index of B, if B is a substring of A else print -1.

Binary String
Difficulty: **Medium**Accuracy: **36.97%**Submissions: **575+**Points: **4**

Given a binary string S. The task is to count the number of substrings that start and end with 1. For example, if the input string is 00100101, then there are three substrings 1001, 100101 and 101.

Input Format:
The first line of input contains T denoting the number of testcases.
Each testcase contains two lines of input. First line contains N, the length of the string. Second line contains the string S.

Output Format:
For each testcase, in a new line, print the answer.

Constraints:

1 <= T <= 100
1 <= N <= 10^4
S is a binary string consisting of only 0 & 1.

Sample Input:
2
4
1111
5
01101
Sample Output:
6
3

Explanation:
Test case 1: There are 6 substrings from
the given string. They are 11, 11, 11,
111, 111, 1111.

Test case 2: There 3 substrings from the
given string. They are 11, 101, 1101.

Searching

Sorted Array Search

Difficulty: **Basic**Accuracy: **48.03%**Submissions: **249K+**Points: **1**

Given an array **arr[]** sorted in ascending order of size **N** and an integer **K**. Check if K is present in the array or not.

Example 1:

Input:
N = 5, K = 6
arr[] = {1,2,3,4,6}
Output: 1
Exlpanation: Since, 6 is present in
the array at index 4 (0-based indexing),
output is 1.

Example 2:

Input:
N = 5, K = 2
arr[] = {1,3,4,5,6}
Output: -1
Exlpanation: Since, 2 is not present
in the array, output is -1.

Your Task:
You don't need to read input or print anything. Complete the function **searchInSorted()** which takes the sorted array arr[], its size N and the element K as input parameters and returns 1 if K is present in the array, else it returns -1.

Expected Time Complexity: O(Log N)
Expected Auxiliary Space: O(1)

Constraints:
$1 <= N <= 10^6$
$1 <= K <= 10^6$
$1 <= arr[i] <= 10^6$

Square root of a number

Difficulty: **Easy**Accuracy: **54.03%**Submissions: **252K+**Points: **2**

Given an integer **n,** find the square root of n. If **n** is not a perfect square, then return the floor value.

Floor value of any number is the greatest Integer which is less than or equal to that number

Examples:

Input: n = 5
Output: 2
Explanation: Since, 5 is not a perfect square, floor of square_root of 5 is 2.
Input: n = 4
Output: 2
Explanation: Since, 4 is a perfect square, so its square root is 2.

Expected Time Complexity: O(logn)

Expected Auxiliary Space: O(1)

Constraints:

$1 \leq n \leq 10^7$

Peak element

Difficulty: **Basic**Accuracy: **38.86%**Submissions: **459K+**Points: **1**

Given an *0-indexed* array of integers *arr[] of size n*, find its peak element and return it's index. An element is considered to be peak if it's value is greater than or equal to the values of its adjacent elements (*if they exist*).

Note: The output will be 1 if the index returned by your function is correct; otherwise, it will be 0.

Examples :

Input: n = 3, arr[] = {1, 2, 3}

Output: 1

Explanation: If the index returned is 2, then the output printed will be 1. Since arr[2] = 3 is greater than its adjacent elements, and there is no element after it, we can consider it as a peak element. No other index satisfies the same property, so answer will be printed as 0.

Input: n = 7, arr[] = {1, 1, 1, 2, 1, 1, 1}

Output: 1

Explanation: In this case there are 5 peak elements with indices as {0,1,3,5,6}. Returning any of them will give you correct answer.

Your Task:

You don't have to read input or print anything. Complete the function *peakElement()* which takes the array *arr[] and its size n* as input parameters and returns the index of the peak element.

Expected Time Complexity: $O(\log(n))$.

Expected Auxiliary Space: $O(1)$

Constraints:

$1 \le n \le 10^5$

$1 \le arr[i] \le 10^6$

Roof Top

Difficulty: **Easy** Accuracy: **40.48%** Submissions: **56K+** Points: **2**

You are given the heights of consecutive buildings. You can move from the roof of a building to the roof of the next adjacent building. You need to find the maximum number of consecutive steps you can put forward such that you gain an increase in altitude with each step.

Examples:

Input: arr[] = [1, 2, 2, 3, 2]

Output: 1

Explanation: 1, 2, or 2, 3 are the only consecutive buildings with increasing heights thus maximum number of consecutive steps with an increase in gain in altitude would be 1 in both cases.

Input: arr[] = [1, 2, 3, 4]

Output: 3

Explanation: 1 to 2 to 3 to 4 is the jump of length 3 to have a maximum number of buildings with increasing heights, so maximum steps with increasing altitude becomes 3.

Expected Time Complexity: O(n)

Expected Auxiliary Space: O(1)

Constraints:

$1 <= arr.size <= 10^6$

$1 <= arr[i] <= 10^5$

Sorting

Bubble Sort

Difficulty: **Basic**Accuracy: **54.33%**Submissions: **810+**Points: **1**

Given an array **arr** of positive integers of size **N**, print the array in decreasing order. You will be given two functions **sort(arr, n)** and **printArray(arr, n).** Both functions accept array and its size as arguments. Here, sort() is used to sort the array and printArray() to print the array in decreasing order.

Note: Use Optimized BubbleSort to sort the array.

Input:

The input line contains **T**, denoting the number of testcases. Each testcase contains two lines. The first line contains N (size of array), second line contains N space separated elements.

Output:

For each testcase print the array in decreasing order in new line.

User Task:

Since, this is a functional problem you don't have to worry about the input, you just have to complete the functions **sort()** and **printArray()**.

Constraints:

$1 <= T <= 10$

$1 <= N <= 10^3$

$1 <= arr[i] <= 10^3$

Example:

Input:

1

6

50 96 756 65 66 683

Output:
756 683 96 66 50

Explanation:
Testcase 1: Sorting the given array using bubble sort will give 50 66 96 683 756. Thus array in decreasing order will be 756 683 96 66 50.

Insertion Sort

Difficulty: **Easy**Accuracy: **66.61%**Submissions: **161K+**Points: **2**

The task is to complete the **insert()** function which is used to implement Insertion Sort.

Examples:

Input: n = 5, arr[] = { 4, 1, 3, 9, 7}
Output: 1 3 4 7 9
Input: n = 10, arr[] = {10, 9, 8, 7, 6, 5, 4, 3, 2, 1}
Output: 1 2 3 4 5 6 7 8 9 10

Your Task:
You don't have to read input or print anything. Your task is to complete the function **insert()** and **insertionSort()** where **insert()** takes the array, it's size and an index i and **insertionSort()** uses insert function to sort the array in ascending order using insertion sort algorithm.

Expected Time Complexity: O(n*n).
Expected Auxiliary Space: O(1).

Constraints:
1 <= n <= 1000
1 <= arr[i] <= 1000

Merge Sort

Difficulty: **Medium**Accuracy: **54.1%**Submissions: **185K+**Points: **4**

Given an array arr[], its starting position l and its ending position r. Sort the array using merge sort algorithm.

Example 1:

Input:
N = 5
arr[] = {4 1 3 9 7}
Output:
1 3 4 7 9

Example 2:

Input:
N = 10
arr[] = {10 9 8 7 6 5 4 3 2 1}
Output:
1 2 3 4 5 6 7 8 9 10

Your Task:

You don't need to take the input or print anything. Your task is to complete the function **merge()** which takes arr[], l, m, r as its input parameters and modifies arr[] in-place such that it is sorted from position l to position r, and function **mergeSort()** which uses merge() to sort the array in ascending order using merge sort algorithm.

Expected Time Complexity: $O(nlogn)$
Expected Auxiliary Space: $O(n)$

Constraints:

$1 <= N <= 10^5$

$1 <= arr[i] <= 10^5$

Merge three sorted arrays

Difficulty: **Medium**Accuracy: **65.94%**Submissions: **16K+**Points: **4**

Given three sorted arrays **A**, **B** and **C** of size **N**, **M** and **P** respectively. The task is to merge them into a single array which must be sorted in increasing order.

Example 1:

Input:
N = 4, A[] = [1 2 3 4]
M = 5, B[] = [1 2 3 4 5]
P = 6, C[] = [1 2 3 4 5 6]
Output: 1 1 1 2 2 2 3 3 3 4 4 4 5 5 6
Explanation: Merging these three sorted arrays, we have:
1 1 1 2 2 2 3 3 3 4 4 4 5 5 6.

Example 2:

Input:
N = 2, A[] = [1 2]
M = 3, B[] = [2 3 4]
P = 4, C[] = [4 5 6 7]
Output: 1 2 2 3 4 4 5 6 7
Explanation: Merging three sorted arrays, we have: 1 2 2 3 4 4 5 6 7.

Your Task:

This is a function problem. You only need to complete the function **mergeThree()** that takes **A,B,C as parameters**. The function **returns** an **array** or **vector**.

Expected Time Complexity: O(N + M + P)
Expected Auxiliary Space: O(N + M + P) for the resultant array only.

Constraints:
$1 <= N, M, P <= 10^6$
$1 <= A[i], B[i], C[i] <= 10^6$

Union of Two Sorted Arrays

Difficulty: **Medium**Accuracy: **31.39%**Submissions: **291K+**Points: **4**

Given two sorted arrays of size **n** and **m** respectively, find their union. The Union of two arrays can be defined as the **common** and **distinct** elements in the two arrays. Return the elements in **sorted** order.

Example 1:

Input:
n = 5, arr1[] = {1, 2, 3, 4, 5}
m = 5, arr2 [] = {1, 2, 3, 6, 7}
Output:
1 2 3 4 5 6 7
Explanation:
Distinct elements including both the arrays are: 1 2 3 4 5 6 7.

Example 2:

Input:
n = 5, arr1[] = {2, 2, 3, 4, 5}
m = 5, arr2[] = {1, 1, 2, 3, 4}
Output:
1 2 3 4 5
Explanation:
Distinct elements including both the arrays are: 1 2 3 4 5.

Example 3:

Input:
n = 5, arr1[] = {1, 1, 1, 1, 1}
m = 5, arr2[] = {2, 2, 2, 2, 2}
Output:
1 2
Explanation:
Distinct elements including both the arrays are: 1 2.

Your Task:
You do not need to read input or print anything. Complete the function **findUnion()** that takes two arrays **arr1[], arr2[],** and their size **n** and **m** as input parameters and returns a list containing the **union of the two arrays**.

Expected Time Complexity: O(n+m).
Expected Auxiliary Space: O(n+m).

Constraints:
$1 <= n, m <= 10^5$
$-10^9 <= arr1[i], arr2[i] <= 10^9$

Stack

Parenthesis Checker

Difficulty: **Easy** Accuracy: **28.56%** Submissions: **573K+** Points: **2**

Given an expression string **x.** Examine whether the pairs and the orders of {,},(,),[,] are correct in exp.
For example, the function should return 'true' for exp = [()]{}{[()()]()} and 'false' for exp = [(]).

Note: The drive code prints "balanced" if function return true, otherwise it prints "not balanced".

Example 1:

Input:
{([])}
Output:
true
Explanation:
{ ([ ]) }. Same colored brackets can form balanced pairs, with 0 number of unbalanced bracket.

Example 2:

Input:
()
Output:
true
Explanation:
(). Same bracket can form balanced pairs, and here only 1 type of bracket is present and in balanced way.

Example 3:

Input:
([]
Output:
false
Explanation:
([]. Here square bracket is balanced but the small bracket is not balanced and Hence , the output will be unbalanced.

Your Task:
This is a **function** problem. You only need to complete the function **ispar()** that takes a **string** as a **parameter** and returns a boolean

value **true** if **brackets** are **balanced** else **returns false**. The **printing** is done **automatically** by the **driver code**.

Expected Time Complexity: O(|x|)
Expected Auixilliary Space: O(|x|)

Constraints:
1 ≤ |x| ≤ 32000

Stock span problem

Difficulty: **Medium**Accuracy: **43.56%**Submissions: **187K+**Points: **4**

The stock span problem is a financial problem where we have a series of **n** daily price quotes for a stock and we need to calculate the span of stocks price for all **n** days.
The span S_i of the stocks price on a given day **i** is defined as the maximum number of consecutive days just before the given day, for which the price of the stock on the given day is less than or equal to its price on the current day.
For example, if an array of 7 days prices is given as {100, 80, 60, 70, 60, 75, 85}, then the span values for corresponding 7 days are {1, 1, 1, 2, 1, 4, 6}.

Example 1:

Input:
N = 7, price[] = [100 80 60 70 60 75 85]
Output:
1 1 1 2 1 4 6
Explanation:
Traversing the given input span
100 is greater than equal to 100 and there are no more elements behind it so the span is 1,

80 is greater than equal to 80 and smaller than 100 so the span is 1,
60 is greater than equal to 60 and smaller than 80 so the span is 1,
70 is greater than equal to 60,70 and smaller than 80 so the span is 2,
60 is greater than equal to 60 and smaller than 70 so the span is 1,
75 is greater than equal to 60,70,60,75 and smaller than 100 so the span is 4,
85 is greater than equal to 80,60,70,60,75,85 and smaller than 100 so the span is 6.
Hence the output will be 1 1 1 2 1 4 6.

Example 2:

Input:
N = 6, price[] = [10 4 5 90 120 80]
Output:
1 1 2 4 5 1
Explanation:
Traversing the given input span
10 is greater than equal to 10 and there are no more elements behind it so the span is 1,
4 is greater than equal to 4 and smaller than 10 so the span is 1,
5 is greater than equal to 4,5 and smaller than 10 so the span is 2,
90 is greater than equal to all previous elements so the span is 4,
120 is greater than equal to all previous elements so the span is 5,
80 is greater than equal to 80 and smaller than 120 so the span is 1,
Hence the output will be 1 1 2 4 5 1.

User Task:
The task is to complete the function **calculateSpan**() which takes two parameters, an array **price[]** denoting the price of stocks, and an integer **N** denoting the size of the array and number of days. This function finds the span of stock's price for all N days and returns an array of length **N** denoting the span for the i-th day.

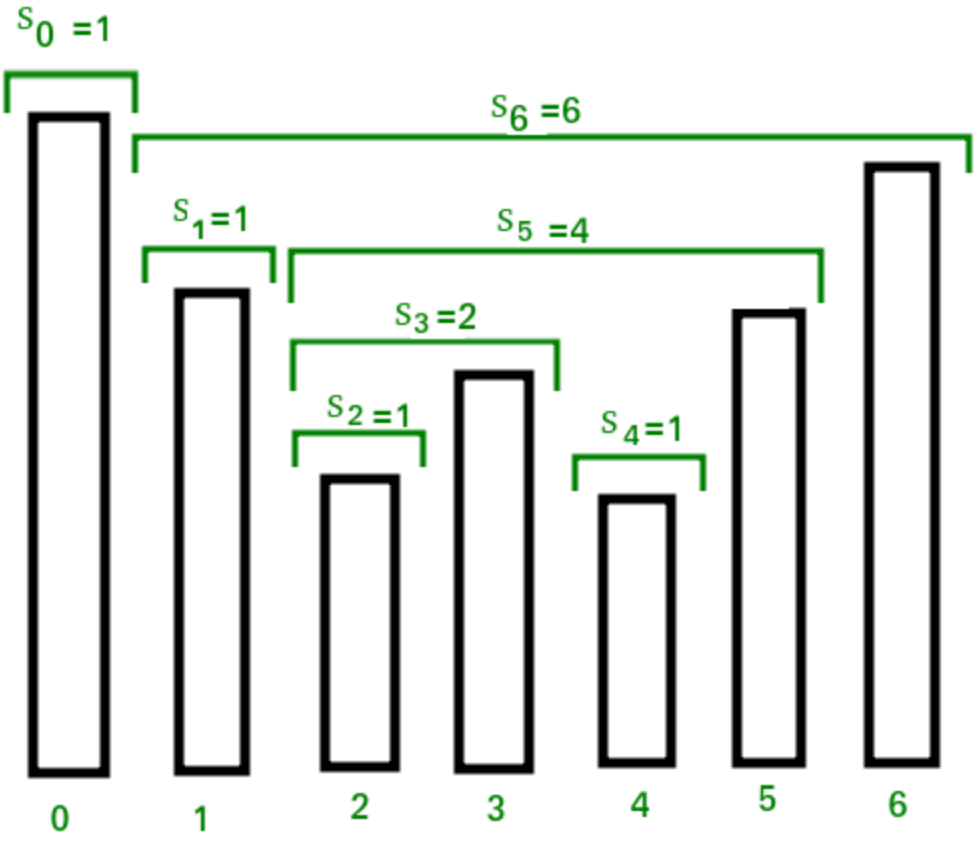
Expected Time Complexity: $O(N)$.

Expected Auxiliary Space: $O(N)$.

Constraints:

$1 \le N \le 10^5$

$1 \le C[i] \le 10^5$



Queue & Deque

Generate Binary Numbers

Difficulty: **Easy**Accuracy: **67.48%**Submissions: **47K+**Points: **2**

Given a number **N**. The task is to generate and print all **binary numbers with decimal values** from **1 to N**.

Example 1:

Input:

N = 2

Output:

1 10

Explanation:

Binary numbers from
1 to 2 are 1 and 10.

Example 2:

Input:

N = 5

Output:

1 10 11 100 101

Explanation:

Binary numbers from
1 to 5 are 1 , 10 , 11 , 100 and 101.

Your Task:

You only need to complete the **function** generate() that
takes **N** as **parameter** and returns vector of strings denoting binary numbers.

Expected Time Complexity : $O(N \log_2 N)$

Expected Auxilliary Space : $O(N \log_2 N)$

Constraints:

$1 \leq N \leq 10^6$

Dequeue Traversal

Difficulty: **Basic**Accuracy: **44.42%**Submissions: **24K+**Points: **1**

Given a Deque **Deq** containing N elements, the task is to traverse the **Deq** and print its elements of it.

Note: The output must be printed on different lines for multiple test cases.

Example 1:

Input:
5
1 2 3 4 5

Output:
1 2 3 4 5

Explanation:
Dqe will look like
{1, 2, 3, 4, 5}.

Example 2:

Input:
1
1

Output:
1

Explanation:
Dqe will look like {1}.

Your task: You don't have to worry about the input. You just have to complete the function **printDeque()** which takes a Deque as an input parameter and prints all its elements of it.

Expected Time Complexity: O(N)

Expected Auxilliary Space: O(1)

Constraints:

$1 \leq N \leq 10^5$

Tree

Height of Binary Tree

Difficulty: **Easy**Accuracy: **78.58%**Submissions: **282K+**Points: **2**

Given a binary tree, find its height.

Example 1:

Input:

```
   1
  / \
 2   3
```

Output: 2

Example 2:

Input:

```
 2
  \
   1
  /
 3
```

Output: 3

Your Task:

You don't need to read input or print anything. Your task is to complete the function **height()** which takes root node of the tree as input parameter and returns an integer denoting the height of the tree. If the tree is empty, return 0.

Expected Time Complexity: O(N)

Expected Auxiliary Space: O(N)

Constraints:

1 <= Number of nodes <= 10^5

1 <= Data of a node <= 10^9

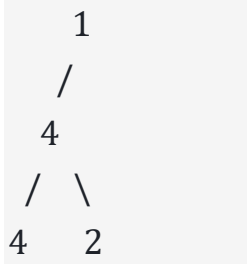
Preorder Traversal

Difficulty: **Basic**Accuracy: **62.73%**Submissions: **162K+**Points: **1**

Given a binary tree, find its preorder traversal.

Example 1:

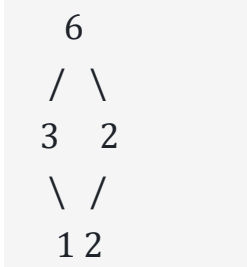
Input:



Output: 1 4 4 2

Example 2:

Input:



Output: 6 3 1 2 2

Your Task:

You just have to complete the function **preorder()** which takes the root node of

the tree as input and returns an array containing the preorder traversal of the tree.

Expected Time Complexity: O(N).

Expected Auxiliary Space: O(N).

Constraints:

1 <= Number of nodes <= 10^4

0 <= Data of a node <= 10^5

Inorder Traversal

Difficulty: **Basic**Accuracy: **67.15%**Submissions: **156K+**Points: **1**

Given a Binary Tree, find the In-Order Traversal of it.

Example 1:

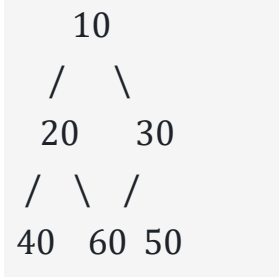
Input:



Output: 3 1 2

Example 2:

Input:



Output: 40 20 60 10 50 30

Your Task:
You don't need to read input or print anything. Your task is to complete the function **inOrder()** that takes root node of the tree as input and returns a list containing the In-Order Traversal of the given Binary Tree.

Expected Time Complexity: O(N).
Expected Auxiliary Space: O(N).

Constraints:
1 <= Number of nodes <= 10^5
0 <= Data of a node <= 10^5

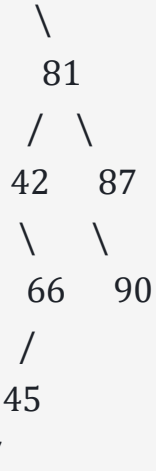
Binary Search Tree

Search a node in BST

Difficulty: **Basic**Accuracy: **68.46%**Submissions: **89K+**Points: **1**

Given a **Binary Search Tree** and a node value X, find if the node with value X is present in the BST or not.

Example 1:

Input:

X = 87

Output: 1

Explanation: As 87 is present in the given nodes , so the output will be 1.

Example 2:

Input:



X = 11

Output: 0

Explanation: As 11 is not present in the given nodes , so the output will be 0.

Your Task:

You don't need to read input or print anything. Complete the function **search()**which returns **true** if the node with **value x** is **present** in the BST**else returns false**.

Expected Time Complexity: O(Height of the BST)

Expected Auxiliary Space: O(1).

Constraints:

1 <= Number of nodes <= 10^5

Delete a node from BST

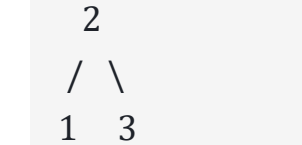
Difficulty: **Medium**Accuracy: **40.7%**Submissions: **119K+**Points: **4**

Given a Binary Search Tree and a node value x. Delete the node with the given value x from the BST. If no node with value x exists, then do not make any change. Return the root of the BST after deleting the node with value x. Do not make any update if there's no node with value x present in the BST.

Note: The generated output will be the inorder traversal of the modified tree.

Examples :

Input:



x = 12

Output: 1 2 3

Explanation: In the given input there is no node with value 12 , so the tree will remain same.

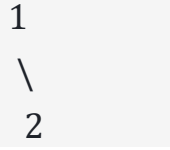
Input:



x = 9

Output: 1 2 4 5 7 8 11 12

Explanation: In the given input tree after deleting 9 will be



```
        \
         8
        / \
       5   11
      / \   \
     4   7   12
```

Expected Time Complexity: O(Height of the BST).

Expected Auxiliary Space: O(Height of the BST).

Constraints:

$1 <= n <= 10^5$

Floor in BST

Difficulty: **Medium**Accuracy: **44.98%**Submissions: **26K+**Points: **4**

Given a Binary search tree and a value X, the task is to complete the function which will return the floor of x.

Note: Floor(X) is an element that is either equal to X or immediately smaller to X. If no such element exits return -1.

Example 1:

Input:

```
    8
   / \
  5   9
 / \   \
2   6   10
```

X = 3

Output: 2

Explanation: Floor of 3 in the given BST is 2

Example 2:

Input:

```
     3
   /   \
  2     5
     /   \
    4     7
   /
  3
```

X = 1

Output: -1

Explanation: No smaller element exits

Your task:

You don't need to worry about the insert function, just complete the function **floor**() which should return the floor of X. If no such element exits return -1.

Expected Time Complexity: O(Height of the BST)

Expected Auxiliary Space: O(Height of the BST).

Constraints:

1 <= Number of nodes <= 10^5

1 <= X, Value of Node <= 10^6

Ceil in BST

Difficulty: **Medium**Accuracy: **62.73%**Submissions: **92K+**Points: **4**

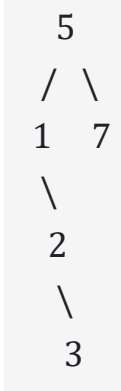
Given a BST and a number **X**, find **Ceil of X**.

Note: Ceil(X) is a number that is either equal to X or is immediately greater than X.

If Ceil could not be found, return -1.

Example 1:

Input:



X = 3

Output: 3

Explanation: We find 3 in BST, so ceil of 3 is 3.

Example 2:

Input:



X = 6

Output: 7

Explanation: We find 7 in BST, so ceil of 6 is 7.

Your task:

You don't need to read input or print anything. Just complete the function **findCeil**() to implement ceil in BST which returns the ceil of **X** in the given **BST.**

Expected Time Complexity: O(Height of the BST)
Expected Auxiliary Space: O(Height of the BST).

Constraints:
1 <= Number of nodes <= 10^5
1 <= Value of nodes<= 10^5

Heap

Binary Heap Operations

Difficulty: **Medium**Accuracy: **22.3%**Submissions: **91K+**Points: **4**

A **binary heap** is a Binary Tree with the following properties:
1) Its a *complete tree* (All levels are completely filled except possibly the last level and the last level has all keys as left as possible). This property of Binary Heap makes them suitable to be stored in an array.

2) A Binary Heap is either **Min Heap** or **Max Heap**. In a *Min Binary Heap*, the key at the *root* must be *minimum* among all keys present in Binary Heap. The same property must be recursively true for all nodes in Binary Tree. Max Binary Heap is similar to MinHeap.

You are given an empty Binary Min Heap and some queries and your task is to implement the three methods **insertKey, deleteKey,** and **extractMin** on the Binary Min Heap and call them as per the query given below:
1) *1 x* (a query of this type means to insert an element in the min-heap with value x)
2) *2 x* (a query of this type means to remove an element at position x from the min-heap)
3) *3* (a query like this removes the min element from the min-heap and prints it).

Example 1:

Input:

Q = 7

Queries:

insertKey(4)
insertKey(2)
extractMin()
insertKey(6)
deleteKey(0)
extractMin()
extractMin()

Output: 2 6 - 1

Explanation: In the first test case for
query
insertKey(4) the heap will have {4}
insertKey(2) the heap will be {2 4}
extractMin() removes min element from
heap ie 2 and prints it
now heap is {4}
insertKey(6) inserts 6 to heap now heap
is {4 6}
deleteKey(0) delete element at position 0
of the heap,now heap is {6}
extractMin() remove min element from heap
ie 6 and prints it now the
heap is empty
extractMin() since the heap is empty thus
no min element exist so -1
is printed.

Example 2:

Input:

Q = 5
Queries:

insertKey(8)
insertKey(9)
deleteKey(1)
extractMin()
extractMin()
Output: 8 -1

Your Task:
You are required to complete the 3 methods **insertKey()** which take one argument the value to be inserted, **deleteKey()** which takes one argument the position from where the element is to be deleted and **extractMin()** which returns the minimum element in the heap(-1 if the heap is empty)

Expected Time Complexity: O(Q*Log(size of Heap)).
Expected Auxiliary Space: O(1).

Constraints:
1 <= **Q** <= 10^4
1 <= **x** <= 10^4

Nearly sorted
Difficulty: **Medium**Accuracy: **75.25%**Submissions: **43K+**Points: **4**
Given an array of **n** elements, where each element is at most **k** away from its target position, you need to sort the array optimally.

Example 1:

Input:
n = 7, k = 3
arr[] = {6,5,3,2,8,10,9}
Output: 2 3 5 6 8 9 10
Explanation: The sorted array will be
2 3 5 6 8 9 10

Example 2:

Input:
n = 5, k = 2
arr[] = {3,1,4,2,5}
Output: 1 2 3 4 5

Note: DO NOT use STL sort() function for this question.

Your Task:
You are required to complete the method **nearlySorted()** which takes 3 arguments and returns the sorted array.

Expected Time Complexity : O(nlogk)
Expected Auxilliary Space : O(n)

Constraints:
$1 \leq n \leq 10^6$
$1 \leq k < n$
$1 \leq arr_i \leq 10^7$

Graph

BFS of graph

Difficulty: **Easy**Accuracy: **44.09%**Submissions: **401K+**Points: **2**

Given a directed graph. The task is to do Breadth First Traversal of this graph starting from 0.

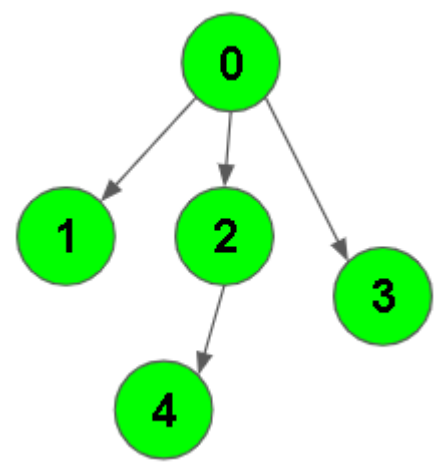
Note: One can move from node u to node v only if there's an edge from u to v. Find the BFS traversal of the graph starting from the 0th vertex, from left to right according to the input graph. Also, you should only take nodes directly or indirectly connected from Node 0 in consideration.

Example 1:

Input:

V = 5, E = 4

adj = {{1,2,3},{},{4},{},{}}



Output:

0 1 2 3 4

Explanation:

0 is connected to 1 , 2 , 3.

2 is connected to 4.

so starting from 0, it will go to 1 then 2

then 3. After this 2 to 4, thus bfs will be

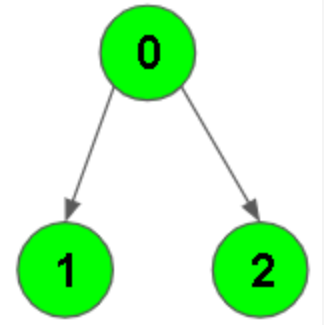
0 1 2 3 4.

Example 2:

Input:

V = 3, E = 2

adj = {{1,2},{},{}}



Output:
0 1 2
Explanation:
0 is connected to 1 , 2.
so starting from 0, it will go to 1 then 2,
thus bfs will be 0 1 2.

Your task:
You dont need to read input or print anything. Your task is to complete the function **bfsOfGraph()** which takes the integer V denoting the number of vertices and adjacency list as input parameters and returns a list containing the BFS traversal of the graph starting from the 0th vertex from left to right.

Expected Time Complexity: $O(V + E)$
Expected Auxiliary Space: $O(V)$

Constraints:
$1 \leq V, E \leq 10^4$

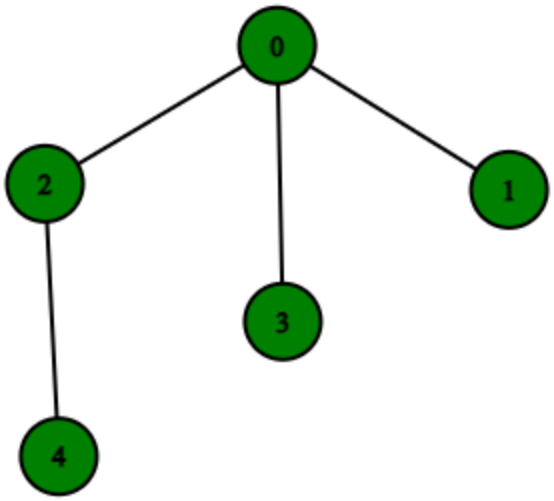
DFS of Graph

Difficulty: **Easy** Accuracy: **63.07%** Submissions: **271K+** Points: **2**

You are given a connected undirected graph. Perform a Depth First Traversal of the graph.
Note: Use the recursive approach to find the DFS traversal of the graph starting from the 0th vertex from left to right according to the graph.

Example 1:

Input: V = 5 , adj = [[2,3,1] , [0], [0,4], [0], [2]]



Output: 0 2 4 3 1

Explanation:

0 is connected to 2, 3, 1.

1 is connected to 0.

2 is connected to 0 and 4.

3 is connected to 0.

4 is connected to 2.

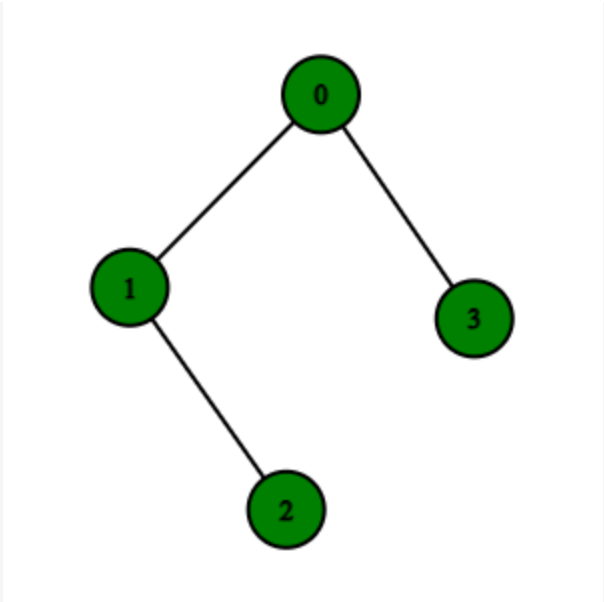
so starting from 0, it will go to 2 then 4,

and then 3 and 1.

Thus dfs will be 0 2 4 3 1.

Example 2:

Input: V = 4, adj = [[1,3], [2,0], [1], [0]]



Output: 0 1 2 3

Explanation:

0 is connected to 1 , 3.

1 is connected to 0, 2.

2 is connected to 1.

3 is connected to 0.

so starting from 0, it will go to 1 then 2

then back to 0 then 0 to 3

thus dfs will be 0 1 2 3.

Your task:

You don't need to read input or print anything. Your task is to complete the function **dfsOfGraph()** which takes the integer V denoting the number of vertices and adjacency list as input parameters and returns a list containing the DFS traversal of the graph starting from the 0th vertex from left to right according to the graph.

Expected Time Complexity: $O(V + E)$

Expected Auxiliary Space: $O(V)$

Constraints:

$1 \leq V, E \leq 10^4$

Greedy

Activity Selection

Difficulty: **Medium** Accuracy: **36.21%** Submissions: **132K+** Points: **4**

Given **N** activities with their start and finish day given in array **start[ ]** and **end[ ]**. Select the maximum number of activities that can be performed by a single person, assuming that a person can only work on a single activity at a given day.

Note : Duration of the activity includes both starting and ending day.

Example 1:

Input:

N = 2

start[] = {2, 1}

end[] = {2, 2}

Output:

1

Explanation:

A person can perform only one of the given activities.

Example 2:

Input:

N = 4

start[] = {1, 3, 2, 5}

end[] = {2, 4, 3, 6}

Output:

3

Explanation:
A person can perform activities 1, 2
and 4.

Your Task :
You don't need to read input or print anything. Your task is to complete the function ***activityselection()*** which takes array **start[ ],** array **end[ ]** and integer **N** as input parameters and returns the maximum number of activities that can be done.

Expected Time Complexity : O(N * Log(N))
Expected Auxilliary Space : O(N)

Constraints:
$1 \leq N \leq 2*10^5$
$1 \leq start[i] \leq end[i] \leq 10^9$

N meetings in one room

Difficulty: **Easy**Accuracy: **45.3%**Submissions: **272K+**Points: **2**

You are given timings of **n** meetings in the form of **(start[i], end[i])** where start[i] is the start time of meeting **i** and end[i] is the finish time of
meeting **i.** Return the **maximum** number of meetings that can be accommodated in a single meeting room, when only one meeting can be held in the meeting room at a particular time.

Note: The start time of one chosen meeting can't be equal to the end time of the other chosen meeting.

Examples :

Input: n = 6, start[] = [1, 3, 0, 5, 8, 5], end[] = [2, 4, 6, 7, 9, 9]
Output: 4

Explanation: Maximum four meetings can be held with given start and end timings. The meetings are - (1, 2), (3, 4), (5,7) and (8,9)

Input: n = 3, start[] = [10, 12, 20], end[] = [20, 25, 30]

Output: 1

Explanation: Only one meetings can be held with given start and end timings.

Expected Time Complexity: O(n*logn)

Expected Auxilliary Space: O(n)

Constraints:

$1 \leq n \leq 10^5$

$0 \leq start[i] < end[i] \leq 10^6$

Backtracking

Rat Maze With Multiple Jumps

Difficulty: **Medium**Accuracy: **37.67%**Submissions: **15K+**Points: **4**

A Maze is given as **N*N** matrix of blocks where source block is the upper left most block i.e., **maze[0][0]** and destination block is lower rightmost block i.e., **maze[N-1][N-1]**. Find if it is possible for the rat to reach from the source block to the destination block. The number of steps rat can jump from (i, j) is represented by maze[i][j].

Note:

If multiple solutions exist, the shortest earliest hop will be accepted. For the same hop distance at any point, forward will be preferred over downward.
In the maze matrix, 0 means the block is the dead end and non-zero number means the block can be used in the path from source to destination. The non-zero value of mat[i][j] indicates number of maximum jumps rat can make from cell mat[i][j].

Example 1:

Input:

N = 4
maze[][] = {{2,1,0,0},{3,0,0,1},
{0,1,0,1},{0,0,0,1}}

Output:

1 0 0 0
1 0 0 1
0 0 0 1
0 0 0 1

Explanation: Rat started with m[0][0] and
can jump up to 2 steps right/down. First
check m[0][1] as it is 1, next check
m[0][2], this won't lead to the solution.
Then check m[1][0], as this is 3(non-zero)
so we can make 3 jumps to reach m[1][3].
From m[1][3] we can move downwards taking
1 jump each time to reach destination at
m[3][3].

Example 2:

Input:

N = 4
maze[][] = {{2,1,0,0}{2,0,0,1},
{0,1,0,1},{0,0,0,1}}

Output: -1

Explanation: As no path exists, so -1.

Your task:

You don't need to read input or print anything. Your task is to complete the function **solve()** which takes the Maze and its dimensions N as inputs and prints the valid path if it exists. If it does not exist, print -1

Note: In the valid path, for each block, if you are visiting the block then you should print 1 for it else 0.

Expected Time Complexity: $O(N^N)$.

Expected Auxiliary Space: O(N) if we don't consider the solution matrix for this.

Constraints:

2 <= n <= 50

0 <= maze[i][j] <= 20

Dynamic Programming

Kadane's Algorithm - I

Difficulty: **Easy**Accuracy: **50.0%**Submissions: **12K+**Points: **2**

Kadane's algorithm comes into picture when we want to find the maximum possible sum in an array when summing the contiguous elements of the array.

You are given an array. Find the **maximum possible sum** of contiguous elements of the array ending at each position in the array. Also, return the overall maximum that we can achieve.

Example 1:

Input:

N = 6

arr[] = {5,-2,-3,32,-5,65}

Output: 5 3 0 32 27 92

92

Explanation: Maximum sum at each index is

5, 3, 0, 32, 27, 92. And, maximum sum for

contiguous array is 92.

Example 2:

Input:

N = 5
arr[] = {-9,-8,8,3,-4}
Output: -9 -8 8 11 7
11

Your Task:
This is a function problem. You don't need to take any input. Just complete the function **maximumSum()** that takes the integer array and its size as inputs and prints the maximum contiguous subarray sum ending at each position in the array. Also, return the overall maximum.

Expected Time Complexity: O(N).
Expected Auxiliary Space: O(N).

Constraints:
1 <= sizeOfArray <= 10^6
-10^3 <= A_i <= 10^3

Kadane's Algorithm - II
Difficulty: **Easy**Accuracy: **40.85%**Submissions: **16K+**Points: **2**
You are given an array **arr** of size **sizeOfArr**. You need to find the **maximum sum** in the array provided that you cannot sum neighboring elements or adjacent elements.

Example 1:

Input:
sizeOfArr = 4
arr[] = {4,5,6,7,8}
Output: 18

Explanation: The given elements are 4 5 6 7 8
For 4, the maximum sum will be 4 as no
element other than 4 from index 0 to 0
For 5, the maximum sum will be 5 as we cannot
add 4 and 5(neighboring elements).
For 6, the maximum sum will be 10 as we can
add 6 and 4.
For 7, the maximum sum will be 12 as we can
add 7 and 5.
For 8, the maximum sum will be 18 as we can
add 4 and 6 and 8.

Example 2:

Input:
sizeOfArr = 5
arr[] = {-9,-8,8,3,-4}
Output: 8

Your Task:
This is a **function** problem. You only need to complete the function **maximumSum()** that takes **array** and **sizeOfArray** and **returns the maximum sum** of the array provided that you cannot sum neighboring elements.

Expected Time Complexity: O(N).
Expected Auxiliary Space: O(N).

Constraints:
$1 \leq sizeOfArr \leq 10^4$
$-10^3 \leq arr_i \leq 10^3$

Longest Common Subsequence

Difficulty: **Medium** Accuracy: **41.68%** Submissions: **280K+** Points: **4**

Given two strings **str1** & **str 2** of length **n** & **m** respectively, return the length of their **longest common subsequence.** If there is no common subsequence then, **return 0**.

A subsequence is a sequence that can be derived from the given string by deleting some or no elements without changing the order of the remaining elements. For example, "abe" is a subsequence of "abcde".

Example 1:

Input: n = 6, str1 = ABCDGH and m = 6, str2 = AEDFHR
Output: 3
Explanation: LCS for input strings “ABCDGH” and “AEDFHR” is “ADH” of length 3.

Example 2:

Input: n = 3, str1 = ABC and m = 2, str2 = AC
Output: 2
Explanation: LCS of "ABC" and "AC" is "AC" of length 2.

Example 3:

Input: n = 4, str1 = XYZW and m = 4, str2 = XYWZ
Output: 3
Explanation: There are two common subsequences of length 3 “XYZ”, and”XYW”, and no common subsequence. of length more than 3.

Your Task:
You do not need to read input or print anything. Complete the function **lcs()** which takes the two strings and their lengths as input parameters and returns the length of the Longest Common Subsequence.

Expected Time Complexity: O(n*m)
Expected Auxiliary Space: O(n*m)

Constraints:

$1 <= n, m <= 10^3$

str1 and str2 are in uppercase alphabet

Longest Increasing Subsequence

Difficulty: **Medium**Accuracy: **32.8%**Submissions: **303K+**Points: **4**

Given an **array a[ ] of n integers**, find the **Length** of the **Longest Strictly Increasing Subsequence**.

A sequence of numbers is called "strictly increasing" when each term in the sequence is smaller than the term that comes after it.

Example 1:

Input: n = 6, a[ ] = {5,8,3,7,9,1}

Output: 3

Explanation: There are more than one **LIS** in this array.
One such Longest increasing subsequence is {5,7,9}.

Example 2:

Input: n = 16, a[ ] = {0,8,4,12,2,10,6,14,1,9,5,13,3,11,7,15}

Output: 6

Explanation: There are more than one LIS in this array.
One such Longest increasing subsequence is {0,2,6,9,13,15}.

Your Task:

You do not need to read input or print anything. Complete the function **longestSubsequence()** which takes the given array and its size as input parameters and returns the **length** of the **longest increasing subsequence.**

Expected Time Complexity : O(n*log(n))

Expected Auxiliary Space: O(n)

Constraints:

$1 \le n \le 10^4$

$0 \le a[i] \le 10^6$

Trie

Search Query for Strings

Difficulty: **Easy**Accuracy: **78.1%**Submissions: **6K+**Points: **2**

Trie is an efficient information retrieval data structure. Use this data structure to store strings and search strings. In this problem, you are given an array of strings (consisting of lowercase characters) **arr**[] of size **N**. Also, you will be given some queries **Q** and for each query a string is given and your task is to check if the given string is in the string array or not.

Note: Each word in the array of string can be of size 10^3.

Example 1:

Input:

N = 8, Q = 3

words[] = {the,a,there,answer,any,by,bye their}

Queries[] = {the,an,any}

Output:

1

0

1

Explanation: After inserting words in the array,all the words will be stored. Now searching for the will result in 1 (present), an will resultin 0(not present) and any will result in 1 (present).

Your Task:
Complete **insert** and **search,** return **true** if the given string is present in the TRIE, else **false** in search function. 1 is printed by the driver's code if the value returned is true, 0 otherwise.

Expected Time Complexity: O(N).
Expected Auxiliary Space: O(N).
N = length of the string

Constraints:
$1 <= N <= 10^3$
$1 <= Q <= 10^3$

Camel Case

Difficulty: **Medium**Accuracy: **46.2%**Submissions: **10K+**Points: **4**

Given a dictionary of words **dict** where each word follows CamelCase notation, print all words in the dictionary that match with a given **pattern** consisting of uppercase characters only.

CamelCase is the practice of writing compound words or phrases such that each word or abbreviation begins with a capital letter. Common examples include: "PowerPoint" and "WikiPedia", "GeeksForGeeks", "CodeBlocks", etc.

Note: The order should be such that the output strings should be sorted by the lexicographic order of their abbreviations. For strings with same abbreviations the strings should be sorted by the usual lexicographic order. So, if Output Strings are Hi and HelloWorld, Hi should come first as H lexiographically is smaller than HW.

Example 1:

Input:

n = 8
dict[] = {Hi,Hello,HelloWorld,HiTech
HiGeek,HiTechWorld,HiTechCity,HiTechLab}
pattern = HA
Output: No match found

Example 2:

Input:
n = 3
dict[]={WelcomeGeek,WelcomeToGeeksForGeeks
GeeksForGeeks}
pattern = WTG
Output: WelcomeToGeeksForGeeks
Explanation: WTG occurs in
Welcome**T**o**G**eeksForGeeks.

Your Task:
Complete **findAllWords()** function and print as mentioned. If the pattern is not found, print "**No match found**" (without quotes) in the function itself. The new line is provided by the driver code.

Expected Time Complexity: O(N * WORD_LEN).
Expected Auxiliary Space: O(N * WORD_LEN).

Constraints:
1 <= N <= 100
1 <= length of pattern <= length of string <= 100

Segment and Binary Indexed Tree

Range Min Max Queries

Difficulty: **Medium**Accuracy: **52.73%**Submissions: **6K+**Points: **4**

You are given an array of size **N** and **Q** queries.

getMinMax(L,R): return the minimum and maximum in the given range [L,R]
updateValue(index,value): update arr[index] to value.

Note: 0-based indexing is used.

Example 1:

Input:
N = 6, Q = 3
arr[] = {11,3,7,5,9,1}
Queries = getMinMax(0,2)
updateValue(3,17)
getMinMax(0,5)

Output:
3 11
1 17

Explanation: There are 3 queries:
Query 1 : Min(0, 1, 2) = 3,
Max(0, 1, 2) = 11
Query 2 : 5 changes to 17
Query 3 : Min(0, 1, 2, 3, 4, 5) = 1,
Max(0, 1, 2, 3, 4, 5) = 17

Your Task:
Complete **updateValue()** and **getMinMax()** function. The **segment tree has been prebuilt** and provided to you.

Expected Time Complexity: O(Q*Log(N)).
Expected Auxiliary Space: O(1).

Constraints:

$1 <= N <= 10^5$

$1 <= Q <= 10^5$

0 <= L, R, index <= N-1

$1 <= arr[i], value <= 10^5$

Range Sum Queries

Difficulty: **Medium**Accuracy: **56.54%**Submissions: **7K+**Points: **4**

You are given an array **arr**[] of size **N** and **Q** queries.

getSum(L,R): return the sum of range [L,R]

updateValue(index,value): update arr[index] to value.

You need to calculate the answer for Type-1 queries and do the update in the segment tree for Type-2 queries.

Note: 0-based indexing is used.

Example 1:

Input:

N = 6, Q = 3

arr[] = {1,3,5,7,9,11}

Queries = getSum(0,2)

updateValue(3,17)

getSum(0,5)

Output:

9

46

Explanation: There are 3 queries:

Query 1: 1 + 3 + 5 = 9

Query 2: 7 changes to 17

Query 3: 1 + 3 + 5 + 17 + 9 + 11= 46

Your Task:

Complete the **updateValue()** and **getSum()** function. The formed segment tree is already provided in both the function as a function argument.

updateValue() processes Type-2 query and does the required update in the segment tree.

getSum() returns the answer for Type-1 query.

Expected Time Complexity: O(Q*Log(N)).

Expected Auxiliary Space: O(1).

Constraints:

1 <= **N** <= 10^5

1 <= **Q** <= 10^5

0 <= **L, R, index** <= N-1

1 <= **arr[i], value** <= 10^5

Example:

Input:

1

6 3

1 3 5 7 9 11

G 0 2

U 3 17

G 0 5

Output:

9

46

Explanation:

Testcase 1: There are 3 queries:

Query 1 : 1 + 3 + 5 = 9

Query 2 : 7 changes to 17

Query 3 : 1 + 3 + 5 + 17 + 9 + 11= 46

Disjoint Set

Union-Find

Difficulty: **Easy**Accuracy: **60.8%**Submissions: **21K+**Points: **2**

This problem is to implement disjoint set union. There will be 2 incomplete functions namely union and find. You have to complete these functions.

Union: Join two subsets into a single set.
isConnected: Determine which subset a particular element is in. This can be used for determining if two elements are in same subset.

Example 1:

Input:
N = 5
q = 4
Queries =
Union(1,3)
isConnected(1,2)
Union(1,5)
isConnected(3,5)
Output:
0
1
Explanation: Initially all nodes 1 2 3 4 5 are not connected.
After **Union(1,3)**, node 1 and 3 will be connected.
When we do **isConnected(1,2)**, as node 1 and 2 are not connected it will return 0.

After **Union(1,5)**, node 1 and 5 will be connected.
When we do **isConnected(3,5)**, as node 1 and 3 are connected, and node 1 and 5 are connected, hence 3 and 5 are connected. Thus it will return 1.

Example 2:

Input:
N = 5
q = 4
Queries =
Union(1,4)
Union(1,5)
isConnected(2,3)
Union(3,4)
Output: 0

Your Task:

You have to complete the function **union_()** which merges the node1 and node2. You will also have to complete the function **isConnected()** which will return whether the two nodes are connected.

Note: Both function will contain two arrays **par[] and ranks1[]** along with two nodes. Initially **par[i] = i** and **rank1[i] = 1**

Expected Time Complexity: O(N + Q).
Expected Auxiliary Space: O(1).

Constraints:
$1 <= N <= 10^5$
$1 <= Q <= 10^5$
1 <= node1, node2 <= N

Detect Cycle using DSU

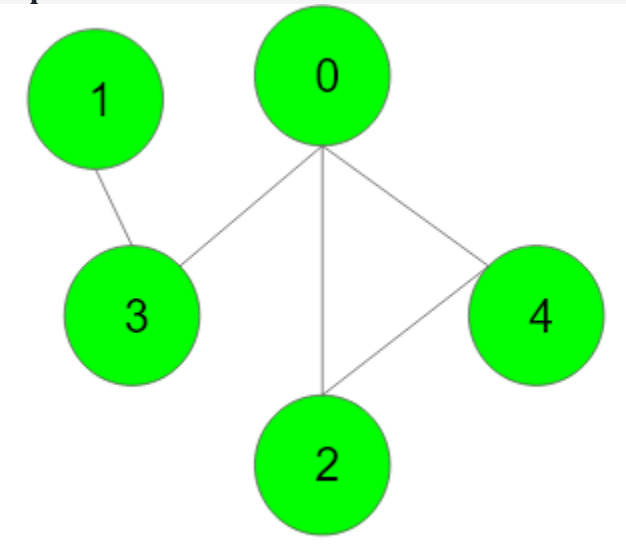
Difficulty: **Medium**Accuracy: **48.37%**Submissions: **58K+**Points: **4**

Given an **undirected graph** with **no self loops** with **V (from 0 to V-1)** nodes and **E** edges, the task is to check if there is any **cycle** in the **undirected graph**.

Note: Solve the problem using **disjoint set union (DSU).**

Example 1:

Input:

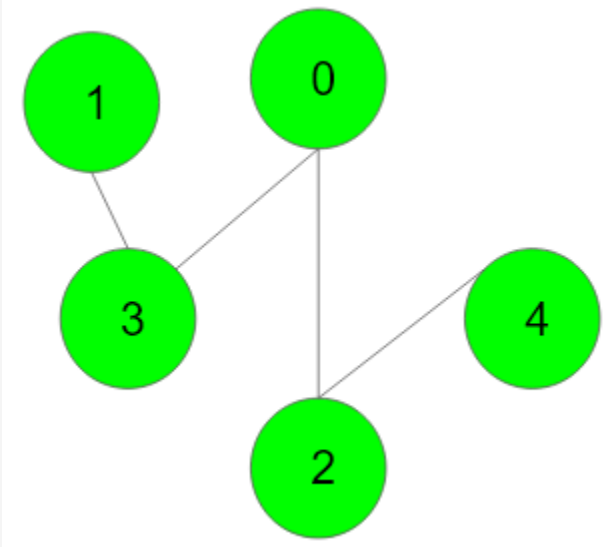


Output: 1

Explanation: There is a cycle between 0->2->4->0

Example 2:

Input:



Output: 0

Explanation: The graph doesn't contain any cycle

Your Task:

You don't need to read or print anyhting. Your task is to complete the function **detectCycle()** which takes number of vertices in the graph denoting as **V** and adjacency list **adj** and returns **1** if graph contains any **cycle** otherwise returns **0**.

Expected Time Complexity: $O(V + E)$

Expected Space Complexity: $O(V)$

Constraints:

$2 \leq V \leq 10^4$

$1 \leq E \leq 10^4$